

CSE331

AUTOMATA AND COMPUTABILITY

Turing machines: encoding, proving decidability, proving undecidability

Spring 2026

Today's learning goals

Sipser 3.1,3.2

- State and use the Church-Turing thesis.
- Explain what it means for a problem to be decidable.
- Justify the use of encoding.
- Give examples of decidable problems.
- Use counting arguments to prove the existence of unrecognizable (undecidable) languages.
- Use diagonalization in a proof of undecidability.

Review

What's $L(M)$?

Is M a decider?

Consider the TM $M =$ "On input w ,

1. Run M_1 on w . If M_1 rejects, rejects. If M_1 accepts, go to 2.
2. Run M_2 on w . If M_2 accepts, accept. If M_2 rejects, reject."

What kind of construction is this?

- A. Formal definition of TM
- B. Implementation-level description of TM
- C. High-level description of TM
- D. I don't know.

Describing TMs

Sipser Section 3.3, p. 184

- **Formal definition:** set of states, input alphabet, tape alphabet, transition function, state state, accept state, reject state.
- **Implementation-level definition:** English prose to describe Turing machine head movements relative to contents of tape.
- **High-level description:** Description of algorithm, without implementation details of machine. As part of this description, can "call" and run another TM as a subroutine.

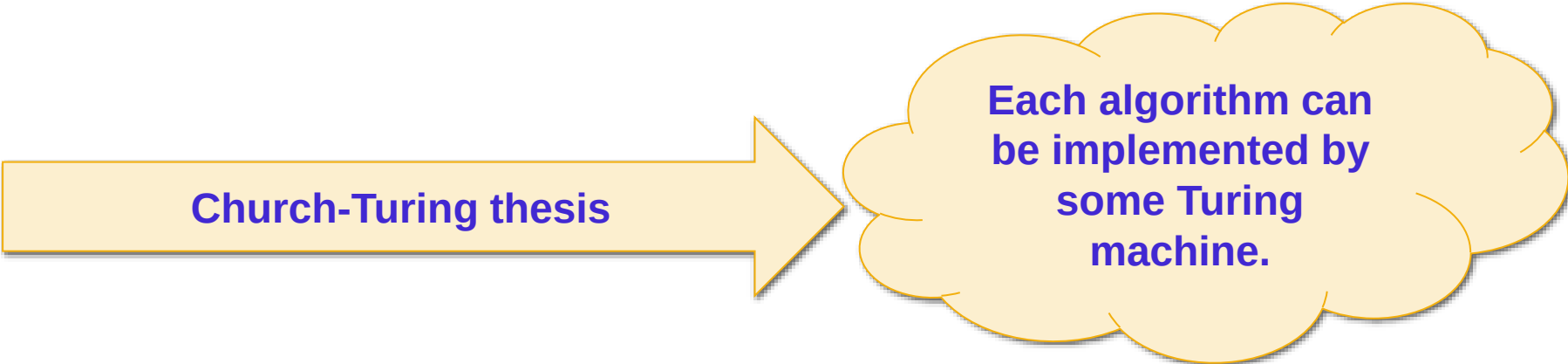
Subroutines

Consider the TM $M =$ "On input w ,

1. Run M_1 on w . If M_1 rejects, rejects. If M_1 accepts, go to 2.
2. Run M_2 on w . If M_2 accepts, accept. If M_2 rejects, reject."

High-level description = Algorithm

- **Wikipedia** "self-contained step-by-step set of operations to be performed"
- "An algorithm is a finite sequence of precise instructions for performing a computation or for solving a problem."



Church-Turing thesis

**Each algorithm can
be implemented by
some Turing
machine.**

At start of CSE 331...

- Pick a model of computation
- Study what problems it can solve
- Prove its limits

Classification: is input of type A or not?

Decision problem

$\{ w \mid w \text{ is of type } A \}$

$\text{PRIME} = \{ 2, 3, 5, 7, \dots \}$

$\text{SORTED} = \{ \langle 1, 3 \rangle, \langle -1, 8, 17 \rangle \dots \}$

Decision problems are coded
by sets of strings

Encoding input for TMs

Sipser p. 159

- By definition, TM inputs are **strings**

- To define TM M:

"On input w ..."

1. ..
2. ..
3. ...

For inputs that aren't strings, we have to **encode the object** (represent it as a string) first

Notation:

$\langle O \rangle$ is the **string** that represents (encodes) the object O

$\langle O_1, \dots, O_n \rangle$ is the **single** string that represents the tuple of objects $O_1, \dots, O_n$

Encoding inputs

Payoff: problems we care about can be reframed as languages of strings

e.g. "Recognize whether a string is a palindrome."

$$\{ w \mid w \text{ in } \{0,1\}^* \text{ and } w = w^R \}$$

e.g. "Check whether a string is accepted by a DFA."

$$\{ \langle B, w \rangle \mid B \text{ is a DFA over } \Sigma, w \text{ in } \Sigma^*, \text{ and } w \text{ is in } L(B) \}$$

e.g. "Check whether the language of a PDA is infinite."

$$\{ \langle A \rangle \mid A \text{ is a PDA and } L(A) \text{ is infinite} \}$$

Example: encoding a DFA

- Think of **encoding** as a **text file** that encodes your data (in some format), that then you store in binary

Example: encoding a DFA

- Think of **encoding** as a **text file** that encodes your data (in some format), that then you store in binary

Example: encode a DFA= $(Q, \Sigma, \delta, q_0, F)$ using JSON

```
{
  "states": [1,2,3,4,5],           // Q={1,2,3,4,5}
  "alphabet": [0,1],              //  $\Sigma=\{0,1\}$ 
  "transition": [((1,0),2), (1,1), 3), ... ], //  $\delta(1,0) = 2, \delta(1,1) = 3, \dots$ 
  "initial_state": 1,             //  $q_0=1$ 
  "accept_states": [4,5],         // F={4,5}
}
```

Computational problems

A computational problem is **decidable** iff the language encoding the problem instances is decidable

Computational problems

Sample computational problems and their encodings:

- A_{DFA} "Check whether a string is accepted by a DFA."
 $\{ \langle B, w \rangle \mid B \text{ is a DFA over } \Sigma, w \text{ in } \Sigma^*, \text{ and } w \text{ is in } L(B) \}$
- E_{DFA} "Check whether the language of a DFA is empty."
 $\{ \langle A \rangle \mid A \text{ is a DFA over } \Sigma, L(A) \text{ is empty} \}$
- EQ_{DFA} "Check whether the languages of two DFAs are equal."
 $\{ \langle A, B \rangle \mid A \text{ and } B \text{ are DFAs over } \Sigma, L(A) = L(B) \}$

FACT: all of these problems are decidable!

Proving decidability

Claim: A_{DFA} is decidable

Proof: WTS that $\{ \langle B, w \rangle \mid B \text{ is a DFA over } \Sigma, w \text{ in } \Sigma^*, \text{ and } w \text{ is in } L(B) \}$ is decidable.

Step 1: construction

How would you check if w is in $L(B)$?

Proving decidability

Define TM M_1 by: $M_1 =$ "On input $\langle B, w \rangle$

1. Check whether B is a valid encoding of a DFA and w is a valid input for B . If not, reject.
2. Simulate running B on w (by keeping track of states in B , transition function of B , etc.)
3. When the simulation ends, by finishing to process all of w , check current state of B : if it is final, *accept*; if it is not, reject."

Proving decidability

Step 1: construction

Define TM M_1 by $M_1 =$ "On input $\langle B, w \rangle$

1. Check whether B is a valid encoding of a DFA and w is a valid input for B . If not, reject.
2. Simulate running B on w (by keeping track of states in B , transition function of B , etc.)
3. When the simulation ends, by finishing to process all of w , check current state of B : if it is final, *accept*; if it is not, *reject*."

Step 2: correctness proof

WTS (1) $L(M_1) = A_{\text{DFA}}$ and (2) M_1 is a decider.

Proving decidability

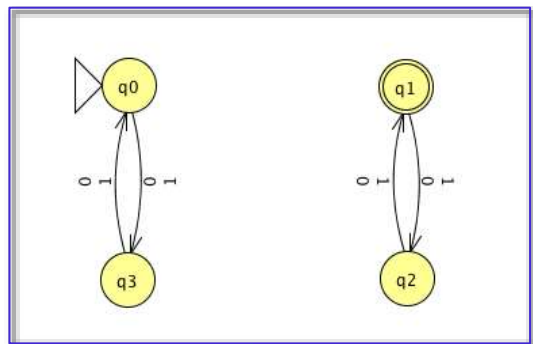
Claim: E_{DFA} is decidable

Proof: WTS that $\{ \langle A \rangle \mid A \text{ is a DFA over } \Sigma, L(A) \text{ is empty} \}$ is decidable.

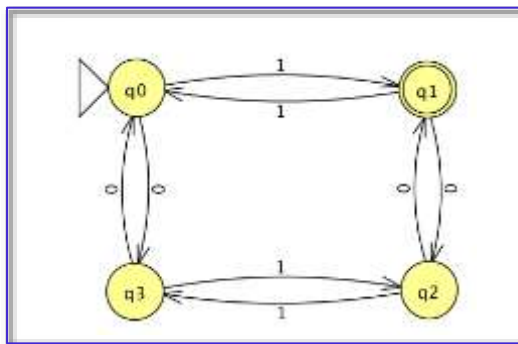
Proving decidability

Claim: E_{DFA} is decidable

Proof: WTS that $\{ \langle A \rangle \mid A \text{ is a DFA over } \Sigma, L(A) \text{ is empty} \}$ is decidable.



should accept

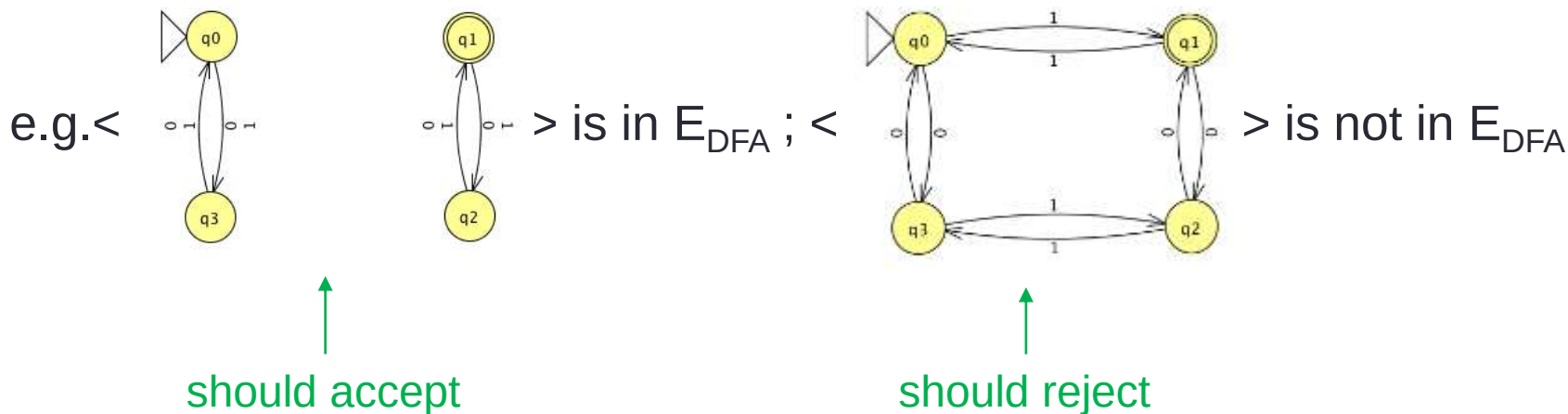


should reject

Proving decidability

Claim: E_{DFA} is decidable

Proof: WTS that $\{ \langle A \rangle \mid A \text{ is a DFA over } \Sigma, L(A) \text{ is empty} \}$ is decidable.



Proving decidability

Claim: E_{DFA} is decidable

Proof: WTS that $\{ \langle A \rangle \mid A \text{ is a DFA over } \Sigma, L(A) \text{ is empty} \}$ is decidable.

Idea: give high-level description

Step 1: construction

*What condition distinguishes between DFA that accept *some* string and those that don't accept *any*?*

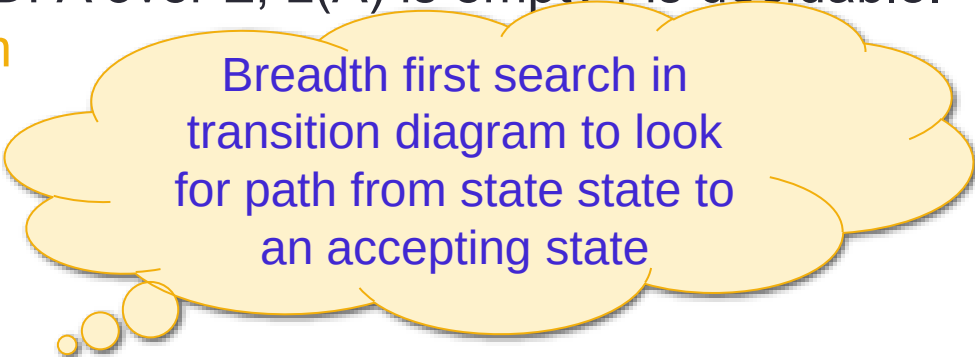
Proving decidability

Claim: E_{DFA} is decidable

Proof: WTS that $\{ \langle A \rangle \mid A \text{ is a DFA over } \Sigma, L(A) \text{ is empty} \}$ is decidable.

Idea: give high-level description

Step 1: construction



Breadth first search in
transition diagram to look
for path from state state to
an accepting state

*What condition distinguishes between DFA that accept *some* string and those that don't accept *any*?*

Proving decidability

Claim: E_{DFA} is decidable

Proof: WTS that $\{ \langle A \rangle \mid A \text{ is a DFA over } \Sigma, L(A) \text{ is empty} \}$ is decidable. **Idea:** give high-level description

Step 1: construction

Define TM M_2 by: $M_2 =$ "On input $\langle A \rangle$:

1. Check whether A is a valid encoding of a DFA; if not, reject.
2. Mark the start state of A .
3. Repeat until no new states get marked:
 - i. Loop over states of A and mark any unmarked state that has an **incoming** edge from a marked state.
4. If no accept state of A is marked, *accept*; otherwise, *reject*.

Proving decidability

Step 1: construction

Define TM M_2 by: $M_2 =$ "On input $\langle A \rangle$:

1. Check whether A is a valid encoding of a DFA; if not, reject.
2. Mark the state state of A .
3. Repeat until no new states get marked:
 - i. Loop over states of A and mark any unmarked state that has an **incoming** edge from a marked state.
4. If no accept state of A is marked, *accept*; otherwise, *reject*.

Step 2: correctness proof

WTS (1) $L(M_2) = E_{\text{DFA}}$ and (2) M_2 is a decider.

Techniques

Sipser 4.1

- **Subroutines**: can use decision procedures of decidable problems as subroutines in other algorithms
 - A_{DFA}
 - E_{DFA}
 - EQ_{DFA}
- **Constructions**: can use algorithms for constructions as subroutines in other algorithms
 - Converting DFA to DFA recognizing complement (or Kleene star).
 - Converting two DFA/NFA to one recognizing union (or intersection, concatenation).
 - Converting NFA to equivalent DFA.
 - Converting regular expression to equivalent NFA.
 - Converting DFA to equivalent regular expression.

Computational problems

A computational problem is **decidable** iff the language encoding the problem instances is decidable

Computational problems

Sample computational problems and their encodings:

- A_{DFA} "Check whether a string is accepted by a DFA."
 $\{ \langle B, w \rangle \mid B \text{ is a DFA over } \Sigma, w \text{ in } \Sigma^*, \text{ and } w \text{ is in } L(B) \}$
- E_{DFA} "Check whether the language of a DFA is empty."
 $\{ \langle A \rangle \mid A \text{ is a DFA over } \Sigma, L(A) \text{ is empty} \}$
- EQ_{DFA} "Check whether the languages of two DFAs are equal."
 $\{ \langle A, B \rangle \mid A \text{ and } B \text{ are DFA over } \Sigma, L(A) = L(B) \}$

FACT: all of these problems are decidable!

Computational problems

Sample computational problems and their encodings:

- A_{PDA} "Check whether a string is accepted by a PDA."
 $\{ \langle B, w \rangle \mid B \text{ is a PDA over } \Sigma, w \text{ in } \Sigma^*, \text{ and } w \text{ is in } L(B) \}$
- E_{PDA} "Check whether the language of a PDA is empty."
 $\{ \langle A \rangle \mid A \text{ is a PDA over } \Sigma, L(A) \text{ is empty} \}$
- EQ_{PDA} "Check whether the languages of two PDAs are equal."
 $\{ \langle A, B \rangle \mid A \text{ and } B \text{ are PDA over } \Sigma, L(A) = L(B) \}$

FACT: some of these problems are decidable, and some are not!

Computational problems

Sample computational problems and their encodings:

- A_{TM} "Check whether a string is accepted by a TM."
 $\{ \langle B, w \rangle \mid B \text{ is a TM over } \Sigma, w \text{ in } \Sigma^*, \text{ and } w \text{ is in } L(B) \}$
- E_{TM} "Check whether the language of a TM is empty."
 $\{ \langle A \rangle \mid A \text{ is a TM over } \Sigma, L(A) \text{ is empty} \}$
- EQ_{TM} "Check whether the languages of two TMs are equal."
 $\{ \langle A, B \rangle \mid A \text{ and } B \text{ are TM over } \Sigma, L(A) = L(B) \}$

FACT: all of these problems are undecidable!

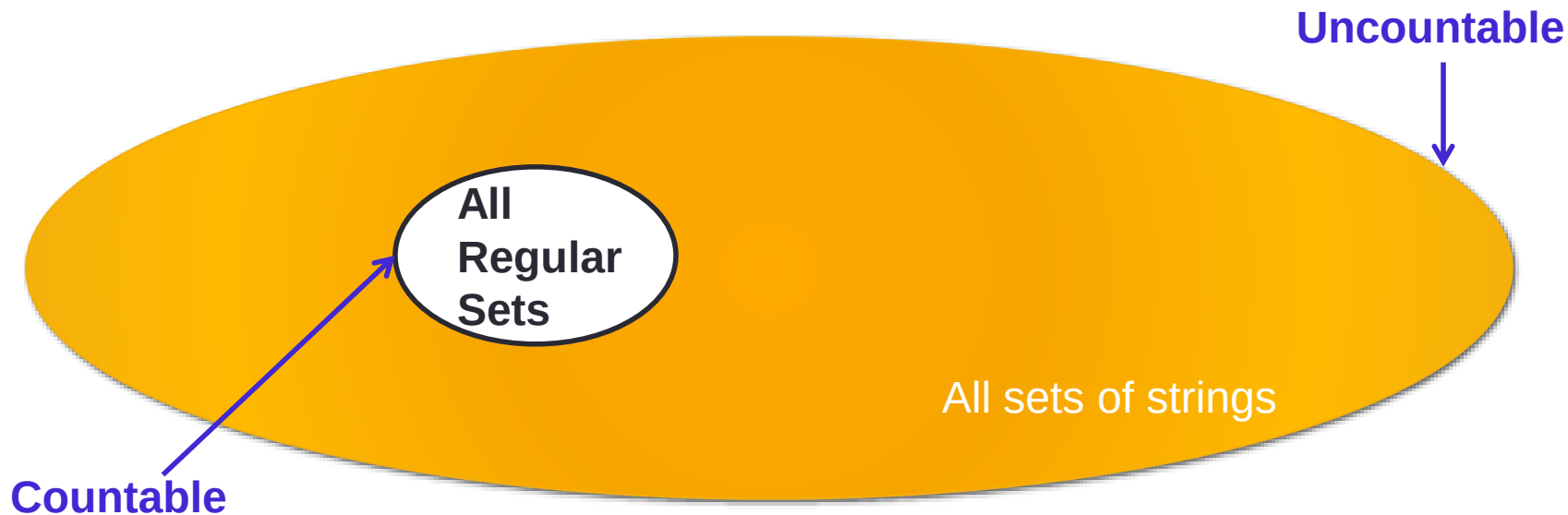
Undecidable?

- There are many ways to prove that a problem *is* decidable.
- How do we find (and prove) that a problem *is not* decidable?

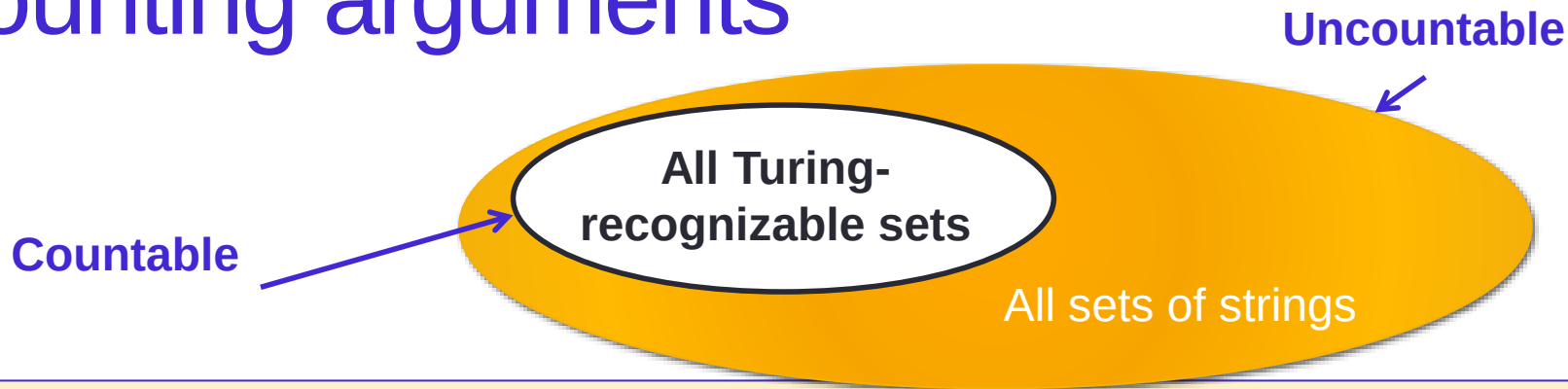
Counting arguments

Before we proved the Pumping Lemma ...

We proved there was a set that was not regular because



Counting arguments



Why is the set of Turing-recognizable languages **countable**?

- A. It's equal to the set of all TMs, which we showed is countable.
- B. It's a subset of the set of all TMs, which we showed is countable.
- C. Each Turing-recognizable language is associated with a TM, so there can be no more Turing-recognizable languages than TMs.
- D. More than one of the above.
- E. I don't know.

Satisfied?

- Maybe not ...
- What's a specific example of a language that is not Turing-recognizable? or not Turing-decidable?
- *Idea: consider a set that, were it to be Turing-decidable, would have to "talk" about itself, and contradict itself!*

A_{TM}

Recall $A_{DFA} = \{ \langle B, w \rangle \mid B \text{ is a DFA and } w \text{ is in } L(B) \}$

Decider for this set simulates arbitrary DFA

$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is in } L(M) \}$

Decider for this set simulates arbitrary TMs ???

A_{TM}

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is in } L(M) \}$$

Define the TM N = "On input $\langle M, w \rangle$:

1. Simulate M on w.
2. If M accepts, accept. If M rejects, reject."

A_{TM}

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is in } L(M) \}$$

Define the TM N = "On input $\langle M, w \rangle$:

1. Simulate M on w .
2. If M accepts, accept. If M rejects, reject."

What is $L(N)$?

A_{TM}

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is in } L(M) \}$$

Define the TM N = "On input $\langle M, w \rangle$:

1. Simulate M on w .
2. If M accepts, accept. If M rejects, reject."

Does N decide A_{TM} ?

A_{TM}

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } w \text{ is in } L(M) \}$$

Define the TM $N =$ "On input $\langle M, w \rangle$:

1. Simulate M on w .
2. If M accepts, accept. If M rejects, reject."

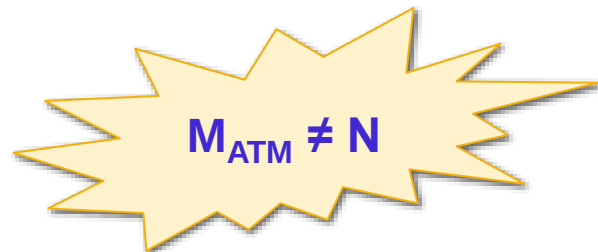
Conclusion: A_{TM} is Turing-recognizable.

Is it decidable?

Diagonalization proof: A_{TM} not decidable *Sipser 4.11*

Assume, **towards a contradiction**, that it is.

Call M_{ATM} the decider for A_{TM} :



For every TM M and every string w ,

- Computation of M_{ATM} on $\langle M, w \rangle$ halts and accepts if w is in $L(M)$.
- Computation of M_{ATM} on $\langle M, w \rangle$ halts and rejects if w is not in $L(M)$.

Diagonalization proof: A_{TM} not decidable *Sipser 4.11*

Assume, towards a contradiction, that M_{ATM} decides A_{TM}

Define the TM $D =$ "On input $\langle M \rangle$:

1. Run M_{ATM} on $\langle M, \langle M \rangle \rangle$.
2. If M_{ATM} accepts, reject; if M_{ATM} rejects, accept."

Diagonalization proof: A_{TM} not decidable *Sipser 4.11*

Assume, towards a contradiction, that M_{ATM} decides A_{TM}

Define the TM $D =$ "On input $\langle M \rangle$:

1. Run M_{ATM} on $\langle M, \langle M \rangle \rangle$.
2. If M_{ATM} accepts, reject; if M_{ATM} rejects, accept."

Consider **running D on input $\langle D \rangle$** . Because D is a decider:

- **either computation halts and accepts ...**
- **or computation halts and rejects ...**

Diagonalization proof: A_{TM} not decidable *Sipser 4.11*

Assume, towards a contradiction, that M_{ATM} decides A_{TM}

Define the TM $D =$ "On input $\langle M \rangle$:

1. Run M_{ATM} on $\langle M, \langle M \rangle \rangle$.
 2. If M_{ATM} accepts, reject; if M_{ATM} rejects, accept."
- If $D(\langle D \rangle)$ **accepts**, then $M_{ATM}(\langle D \rangle, \langle D \rangle)$ **rejects**, which means $D(\langle D \rangle)$ should have **rejected**
 - If $D(\langle D \rangle)$ **rejects**, then $M_{ATM}(\langle D \rangle, \langle D \rangle)$ **accepts**, which means $D(\langle D \rangle)$ should have **accepted**
 - Either way, we reached a **contradiction**

Diagonalization proof: A_{ATM} not decidable Sipser 4.11

Assume, to

Diagonalization???

Def

Self-reference

1. R

2. If

"Is $\langle D \rangle$ an element of $L(D)$?"

Consider **running on input $\langle D \rangle$** . Because D is a decider:

- **either computation halts and accepts ...**
- **or computation halts and rejects ...**

Why is this called diagonalization?

- Let $M_1, M_2, \dots$ be an enumeration of **all TMs**
- Let $w_1, w_2, \dots$ be an enumeration of **all inputs**
- Construct table: $T[i, j] = \begin{cases} 1 & \text{if } M_i(w_j) \text{ accepts} \\ 0 & \text{otherwise} \end{cases}$

	w_1	w_2	w_3	...
M_1	0	0	1	
M_2	1	0	1	
M_3	0	0	1	
...				

Why is this called diagonalization?

	w_1	w_2	w_3	...
M_1	0	0	1	
M_2	1	0	1	
M_3	0	0	1	
...				

- Rows of table describe **all recognizable languages**
- Any other row (=language) **cannot be recognizable**
- We will construct such a language

Why is this called diagonalization?

- Next idea: specialize only to inputs $\langle M_i \rangle$

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	...
M_1	0	0	1	
M_2	1	0	1	
M_3	0	0	1	
...				

- Suffices to construct row (=language) that **differs from any row on some of these inputs**

Why is this called diagonalization?

- Consider the **diagonal**: running M_i on $\langle M_i \rangle$; then **flip it**

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	...
M_1	0	0	1	
M_2	1	0	1	
M_3	0	0	1	
...				

- $L = \{\langle M \rangle : M \text{ doesn't accept } \langle M \rangle\}$
- L cannot be recognized by any M_i – they disagree on input $\langle M_i \rangle$
- Since we consider all TMs, L must be **un-recognizable**